

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Оглавление

Модули и пакеты.....	2
Что такое модуль в Python?	2
ModuleNotFoundError	2
Установка модулей в PyCharm	3
Удаление модулей в PyCharm	5
Установка модулей через pip	5
Инструкции import и from	6
Создание своего модуля	9
Что такое пакет в Python?	9
Структура пакета:.....	10

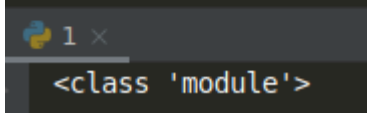
Модули и пакеты

Что такое модуль в Python?

Модуль в Python это файл с расширением .py. Модули предназначены для того, чтобы в них хранить часто используемые функции, классы, константы и т.п. Файлы .py можно условно разделить на модули и программы - программы предназначены для непосредственного запуска, а модули для импортирования их в другие программы. Стоит заметить, что модули могут быть написаны не только на языке Python, но и на других языках (например C).

```
import sys

print(type(sys))
```

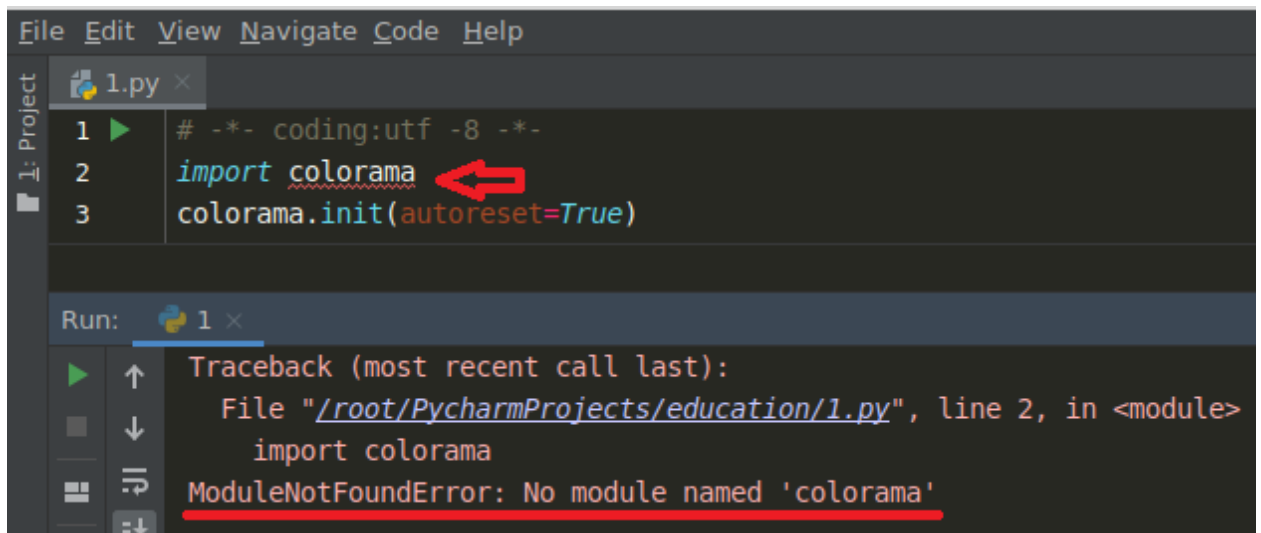


1 x

<class 'module'>

ModuleNotFoundError

В процессе запуска сторонних программ очень часто приходится сталкиваться со следующей ошибкой – [ModuleNotFoundError](#). Это значит, что для работоспособности программы требуется установка зависимости – модуля (библиотеки).



The screenshot shows the PyCharm IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, and Help. The left sidebar shows a project view with a file named 1.py. The main editor window displays the following code:

```
1 # -*- coding:utf-8 -*-
2 import colorama
3 colorama.init(autoreset=True)
```

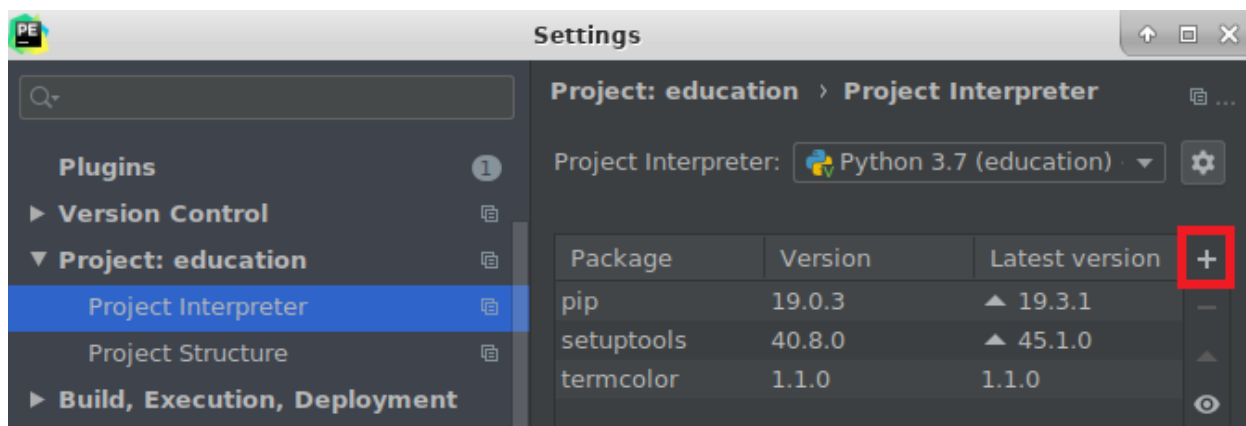
A red arrow points to the `import colorama` line. Below the code editor, the Run tab is active, showing a traceback (most recent call last):

```
File "/root/PycharmProjects/education/1.py", line 2, in <module>
    import colorama
ModuleNotFoundError: No module named 'colorama'
```

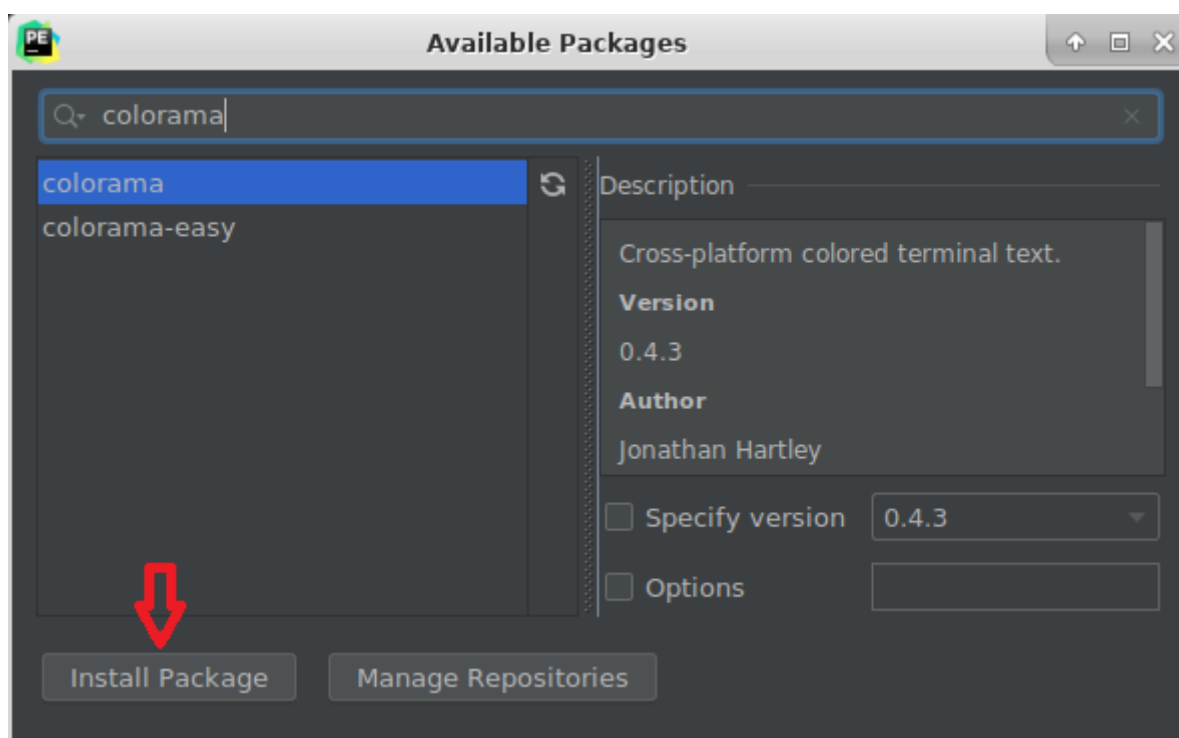
The error message is underlined in red.

Установка модулей в PyCharm

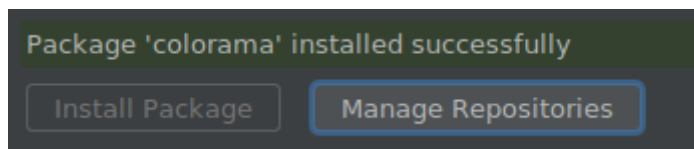
Для установки модуля нужно зайти в меню **File** → **Settings** → Откроется окно настроек, в котором нужно раскрыть вкладку с названием проекта и щёлкнуть по **Project Interpreter**. Далее нажать на значок плюс.



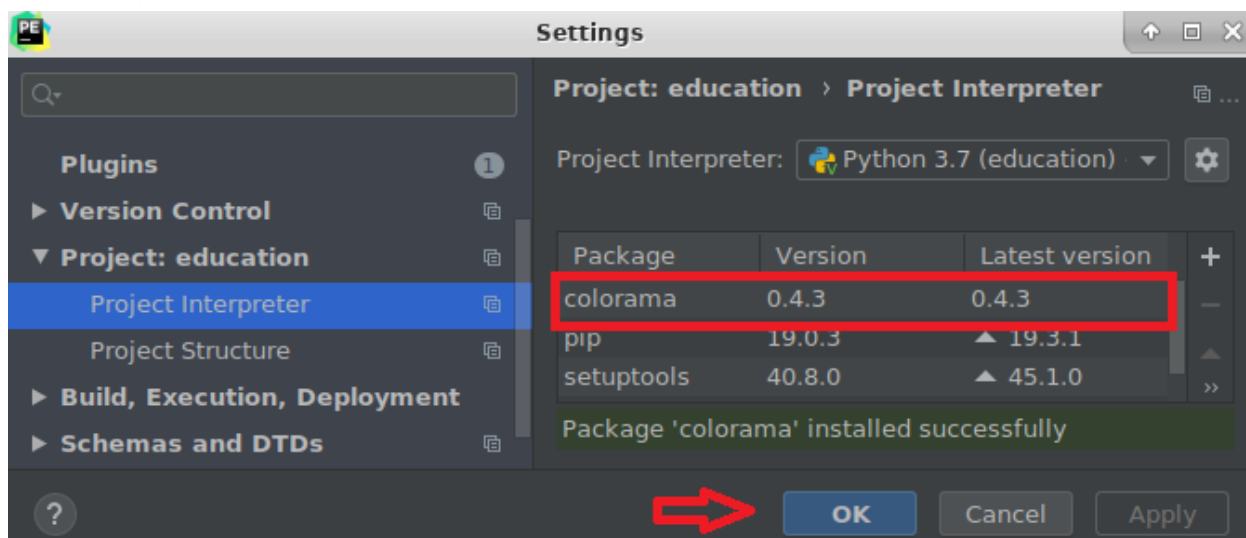
В открывшемся окне, в поисковой строке ввести имя модуля, и если он найден, для установки нажать кнопку **Install Package**.



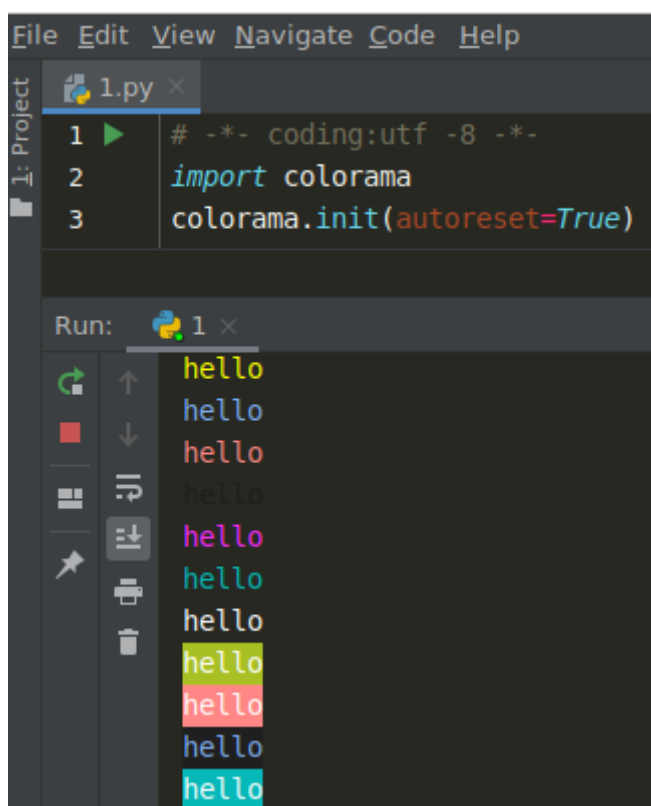
При успешной установке, выведется подтверждающее сообщение.



После чего окно установки модулей нужно закрыть, и нажать кнопку ОК в окне настроек, где уже видно, что добавился установленный модуль.

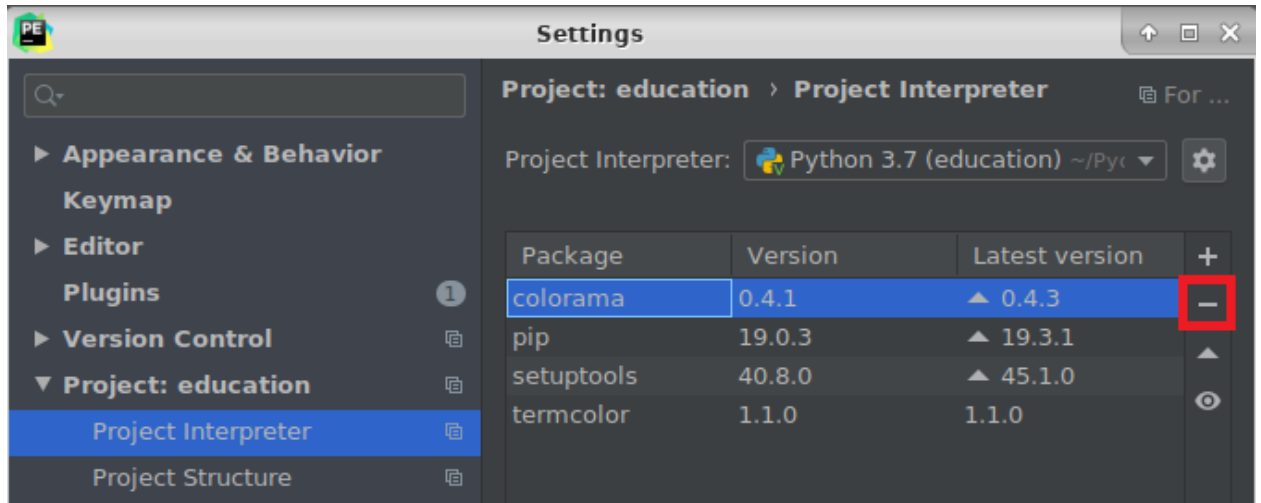


После установки, пакет сразу доступен для использования.

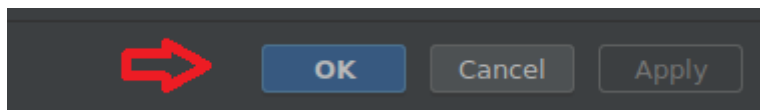


Удаление модулей в PyCharm

Для удаления модуля нужно в файле настроек выделить щелчком мышки нужный модуль, нажать на значок с минусом



и подтвердить внизу окна с настройками, нажав на кнопку OK.



Установка модулей через pip

Обратите внимание, что таким образом мы устанавливаем модуль **в виртуальное окружение проекта**, который создал PyCharm.

```

root@exp:~# pip3 list
Package            Version
-----
AdvancedHTTPServer 2.1.0
aio-ping            0.0.0
aiohttp             3.6.1
ajpy                 0.0.4
alembic             1.0.0.dev0
aniso8601           4.1.0
asn1crypto          0.24.0
async-timeout       3.0.1
asyncio             3.4.3
attrs               19.1.0
backdoor-factory    0.0.0
basemap             1.2.0
bcrypt              3.1.6
beautifulsoup4      4.6.0

```

Для того, чтобы посмотреть список установленных модулей, нужно ввести в терминал команду `pip3 list` или `pip list`.

Для установки нужного модуля, пишем в терминале команду `pip3 install (pip install) <имя модуля>`

```

└─> pip install colorama
Collecting colorama
  Using cached colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Installing collected packages: colorama
Successfully installed colorama-0.4.6

```

Для установки модуля для конкретной версии python используется следующая команда:

```
python<версия> -m pip install <имя модуля>
```

При работе с системным python'ом модули будут устанавливаться в папку `/usr/local/lib/python<версия>/dist-packages`

А модули установленные в Pycharm хранятся в другом месте, в данном случае в `/root/PycharmProjects/education/venv/lib/python<версия>/site-packages/colorama`

Для удаления модуля введите команду `pip uninstall <имя модуля>` Но если мы хотим обновить модуль, то можно сделать это одной командой `pip install --upgrade <имя модуля>`

```

root@exp:~# pip3 install --upgrade colorama
Collecting colorama
  Using cached https://files.pythonhosted.org/packages/6b/31/17e6d5708a08029992b2fee2c143c7a0a5cc5/colorama-0.4.3-py2.py3-none-any.whl (25 kB)
Installing collected packages: colorama
  Found existing installation: colorama 0.3.9
  Uninstalling colorama-0.3.9:
    Successfully uninstalled colorama-0.3.9
  Successfully installed colorama-0.4.3

```

Инструкции import и from

Самый простой способ импортировать модуль в Python это воспользоваться инструкцией `import`. Для этого в начале программы пишем `import <имя модуля>` например `import sys`.

`import sys` - такой способ используется, если в вашем проекте могут использоваться такие же имена переменных или функций, как и в модуле `sys`. При таком подключении, вызов функций или переменных из модуля выполняется так: `sys.<имя функции>`.

```

import sys

print(sys.path[0])

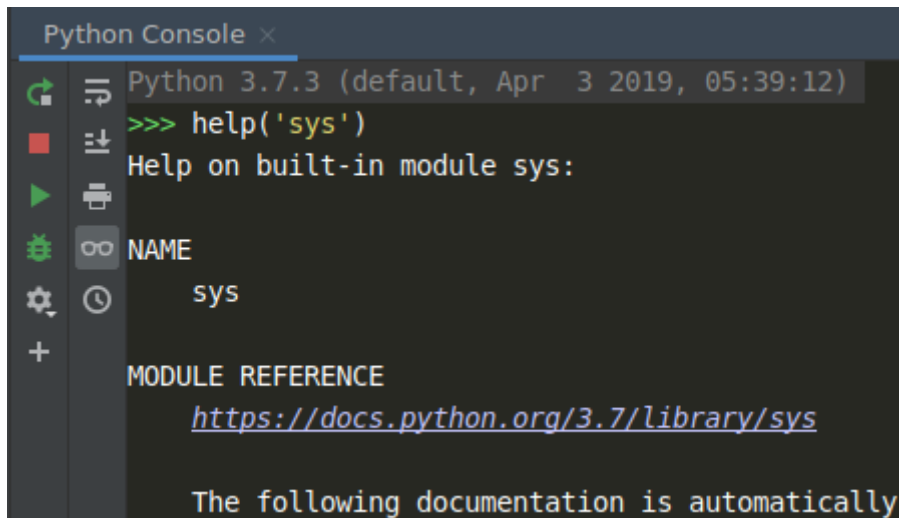
```

1 x

/root/PycharmProjects/education

Модуль `sys` является встроенным, и устанавливать его не требуется. Модуль `sys` имеет много функций, `path` выводит список строк, которые показывают, в каких директориях ищутся модули. Первый элемент списка (`path[0]`) – это директория, в которой находится скрипт, запускающий интерпретатор Python.

Чтобы посмотреть исчерпывающую информацию по модулю, можно в консоли отправить команду `help('sys')`. Справочную информацию можно получить по любому установленному модулю.



```
Python Console x
Python 3.7.3 (default, Apr 3 2019, 05:39:12)
>>> help('sys')
Help on built-in module sys:

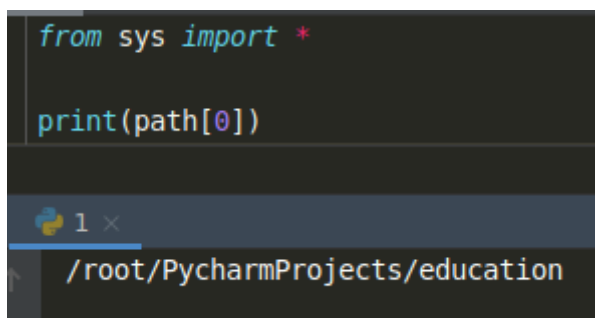
NAME
    sys

MODULE REFERENCE
    https://docs.python.org/3.7/library/sys

The following documentation is automatically
```

Другой способ подключения модуля:

`from sys import *` - способ используется тогда, когда программист уверен, что пересечений имён нет (иначе может возникнуть конфликт). При вызове методов или функций не нужно ставить префикс `sys`, что уменьшает количество кода. Такой подход позволяет импортировать все доступные элементы модуля (переменные, функции, классы), для работы с ними не требуется использовать имя модуля.



```
from sys import *

print(path[0])

1 x
/root/PycharmProjects/education
```

На самом деле, нет никакой необходимости загружать все возможности модуля. Для того чтобы использовать только нужную часть модуля, звёздочку нужно заменить на имена конкретных функций, перечисленные через запятую. Именно этот способ является рекомендуемым.


```
from sys import path

print(path[0])
```

```
1 x
/root/PycharmProjects/education
```

Для наглядности добавим ещё одну функцию `version` модуля `sys`.

`version` - выводит номер версии Python, а также дополнительную информацию о номере сборки и используемом компиляторе.

```
from sys import path, version

print(version, path[0], sep='\n')
```

```
1 x
3.7.3 (default, Apr  3 2019, 05:39:12)
[GCC 8.3.0]
/root/PycharmProjects/education
```

`import sys as s` - при такой записи название «`sys`» заменяется на «`s`». Способ используют, чтобы заменить длинное название модуля на более короткое. Хотя в данном случае это излишне, имя и так короткое.

```
import sys as s

print(s.version, s.path[0], sep='\n')
```

```
1 x
3.7.3 (default, Apr  3 2019, 05:39:12)
[GCC 8.3.0]
/root/PycharmProjects/education
```

Импортируемому объекту можно тоже задать псевдоним.

```
from sys import version as v

print(v)
```

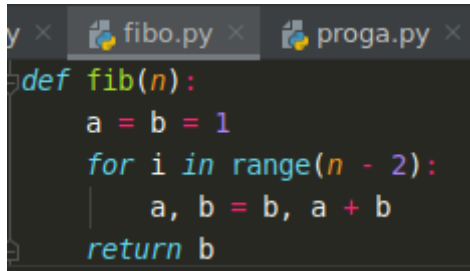
```
1 x
3.7.3 (default, Apr  3 2019, 05:39:12)
[GCC 8.3.0]
```

Все что импортируется из модуля можно получить через функцию `dir()`.

```
import sys
print(dir(sys))
```

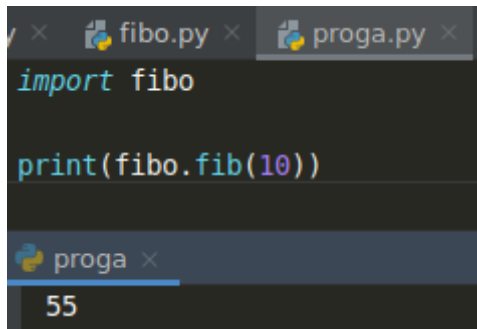
Создание своего модуля

Создадим файл модуля `fibonacci.py` и запишем в него функцию, вычисляющую n -ое число ряда Фибоначчи.



```
def fib(n):  
    a = b = 1  
    for i in range(n - 2):  
        a, b = b, a + b  
    return b
```

И сделаем второй файл `proga.py`, в который мы импортируем модуль `fibonacci`. Теперь мы можем из программы `proga.py` делать расчёты, используя функцию из модуля.



```
import fibo  
  
print(fibo.fib(10))
```

proga ×
55

Или так:

```
from fibo import fib  
print(fib(10))
```

Импорт будет работать, если созданный нами модуль лежит в одном каталоге с программой, либо находится в директории `site-packages`, куда по-умолчанию устанавливаются модули и пакеты.

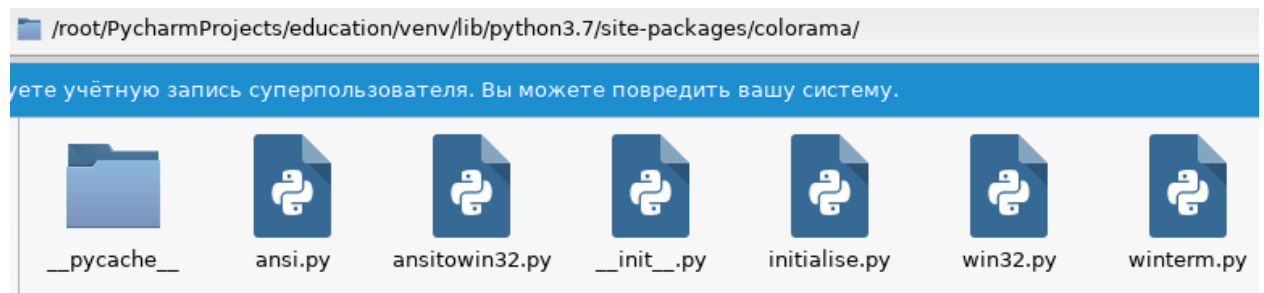
Что такое пакет в Python?

Пакет в Python — это каталог, включающий в себя другие каталоги и модули, но при этом дополнительно содержащий файл `__init__.py`. Пакеты используются для формирования пространства имен, что позволяет работать с модулями через указание уровня вложенности (через точку).

Файл `__init__.py` обычно пуст, или используется для экспорта выбранных частей пакета под более удобными именами, хранения вспомогательных функций и т. д. Однако если вы удалите файл `__init__.py`, Python больше не будет искать подмодули внутри этого каталога, поэтому попытки импортировать модуль будут неудачными.

Мы уже выяснили, что модули в PyCharm ищутся интерпретатором в директории `/root/PycharmProjects/education`. Заглянем в каталог с пакетом `colorama`.

Структура пакета:



Мы видим, что в состав пакета входит целый ряд py файлов.

После того, как модуль был найден, производится его компиляция в байт-код, если это необходимо. Это делается в том случае, если байт-код более старый по сравнению с файлом исходного кода (или запускается в другом интерпретаторе). В Python 3 после компиляции создается каталог с именем `__pycache__`, в нем располагаются файлы с расширением `.pyc`, в них содержится байт-код.